LAB 03
Permissions and Roles

Microsoft Azure SQL Database — SecureDB

Student ID: N01650532

Course: CPAN 363 — Database Security

Lab: Week 4 — Permissions and Roles

Evidence: VS Code Azure SQL query screenshots supplied during lab walkthrough

Submission: PDF report and companion SQL file

Objective

Implement three least-privilege database roles, test their effective permissions through EXECUTE AS USER, demonstrate that an explicit DENY overrides a role-based GRANT, and produce database audit reports.

All employee names and salary amounts are fictional lab data. No passwords or connection secrets are included.

Part 1 — Create and populate tables

Created dbo.Employees for ordinary employee details and dbo.Salaries for confidential compensation records. Inserted three fictional rows in each table. The two SELECT statements returned the expected records.

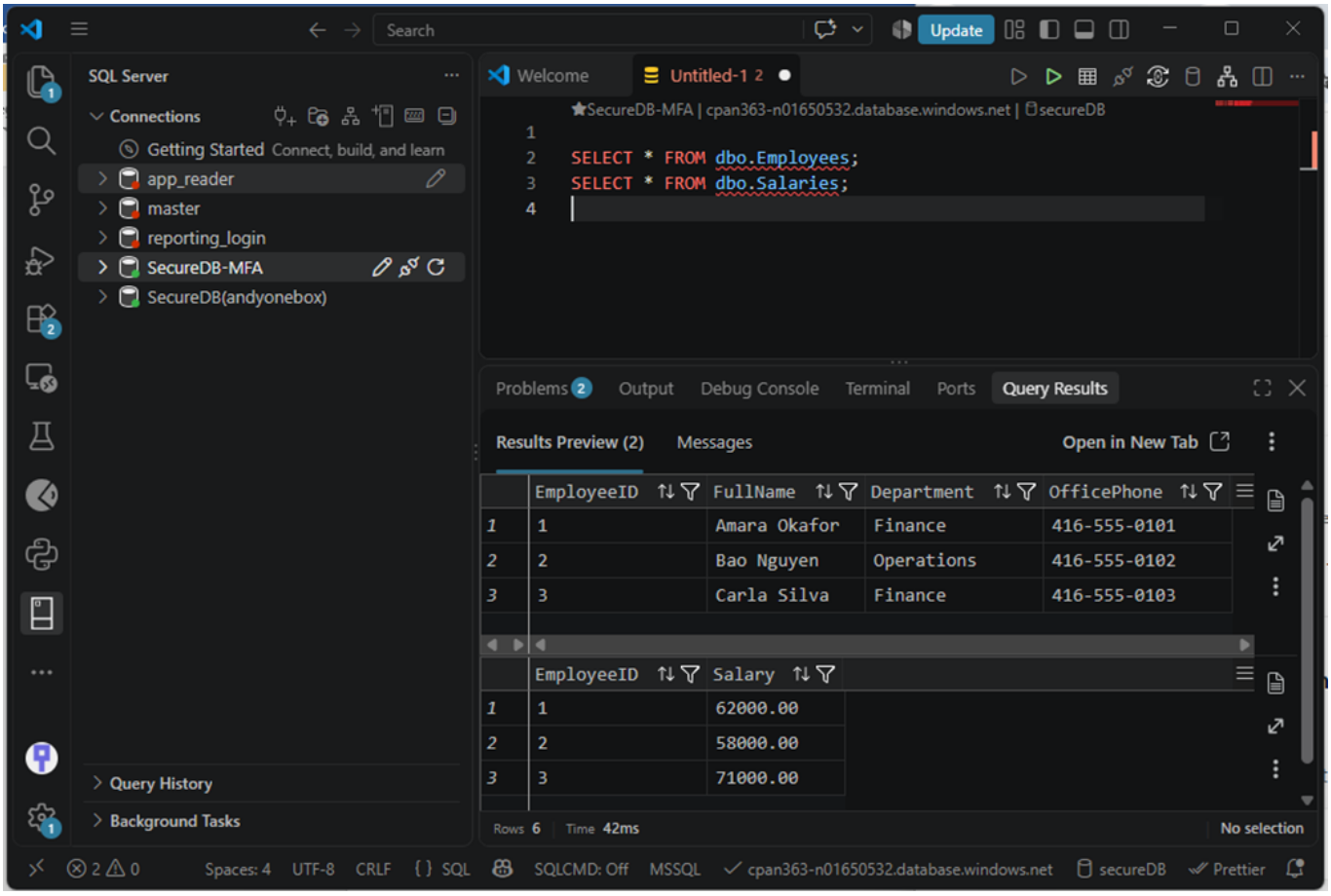


Figure 1. Employees and Salaries tables populated with three sample records each.

Part 2 — Roles, permissions and users

Created support_role, hr_role and analyst_role. Created corresponding WITHOUT LOGIN test users and added each user to its matching role. The role membership query confirmed all three assignments.

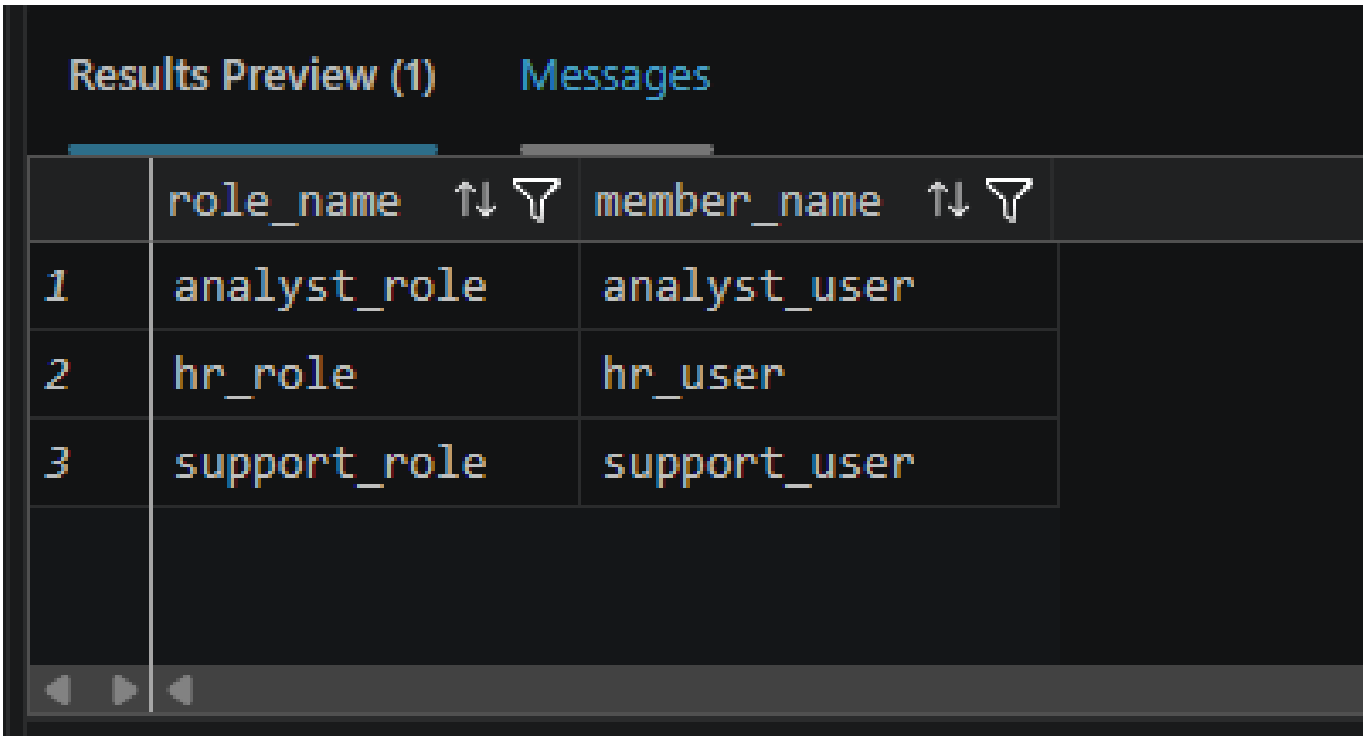


Figure 2. Role membership verification: analyst_user, hr_user and support_user.

Part 3 — Effective permission tests

Support: HAS_PERMS_BY_NAME returned 1 for Employees SELECT and 0 for Salaries SELECT. Reading Employees succeeded; reading Salaries returned SQL Server error 229.

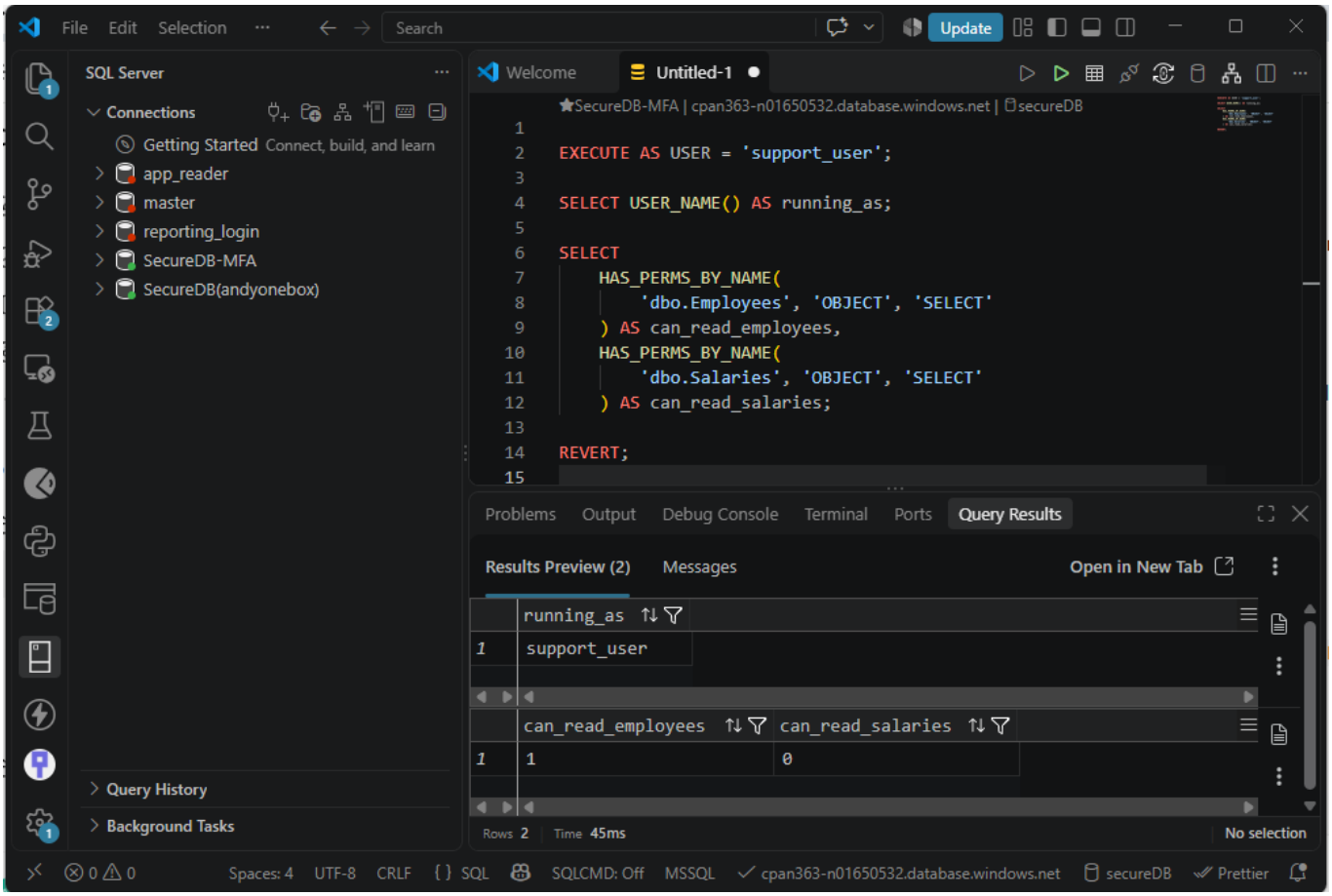


Figure 3. Support effective permission values: Employees = 1, Salaries = 0.

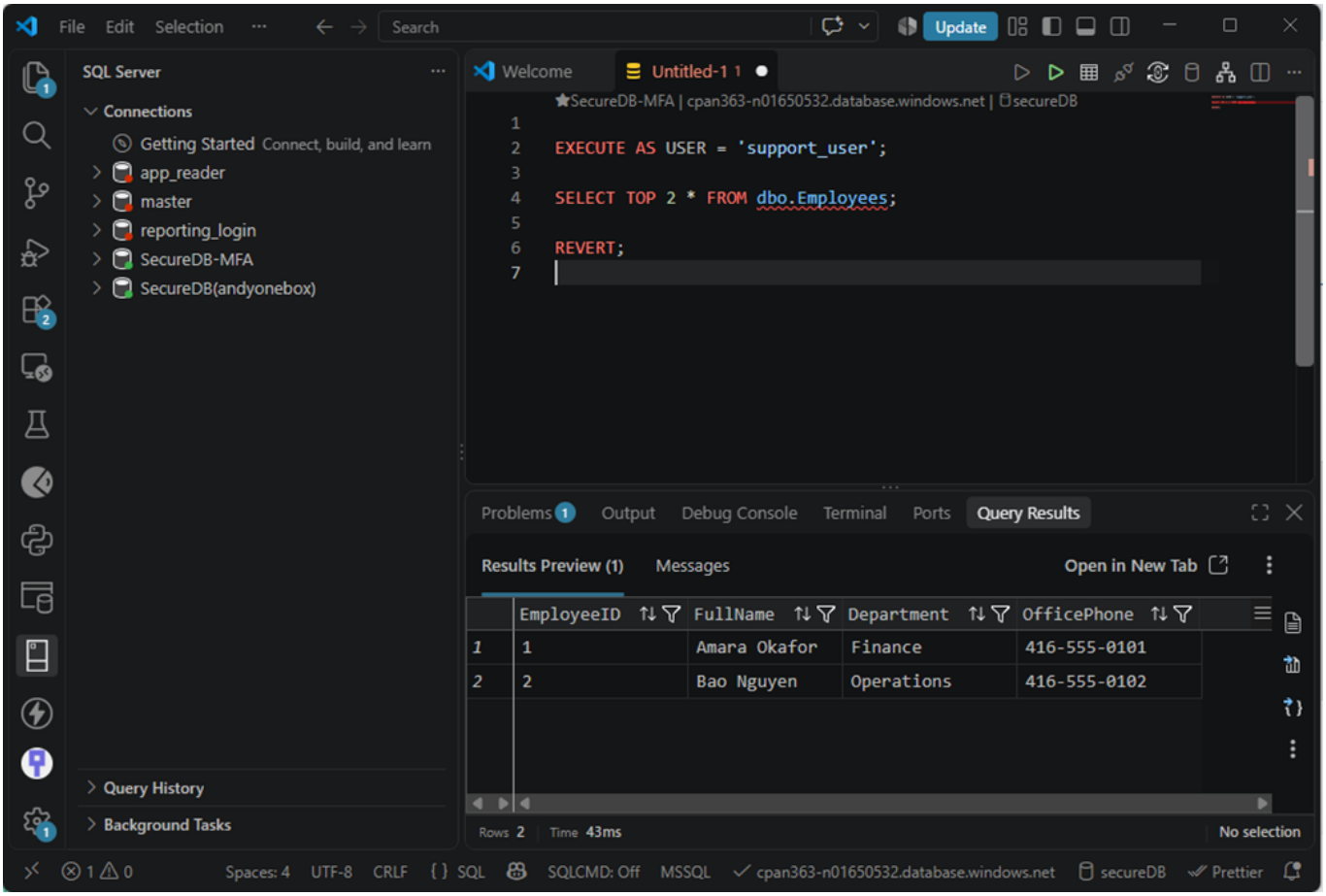


Figure 4. Support user successfully reads two employee rows.

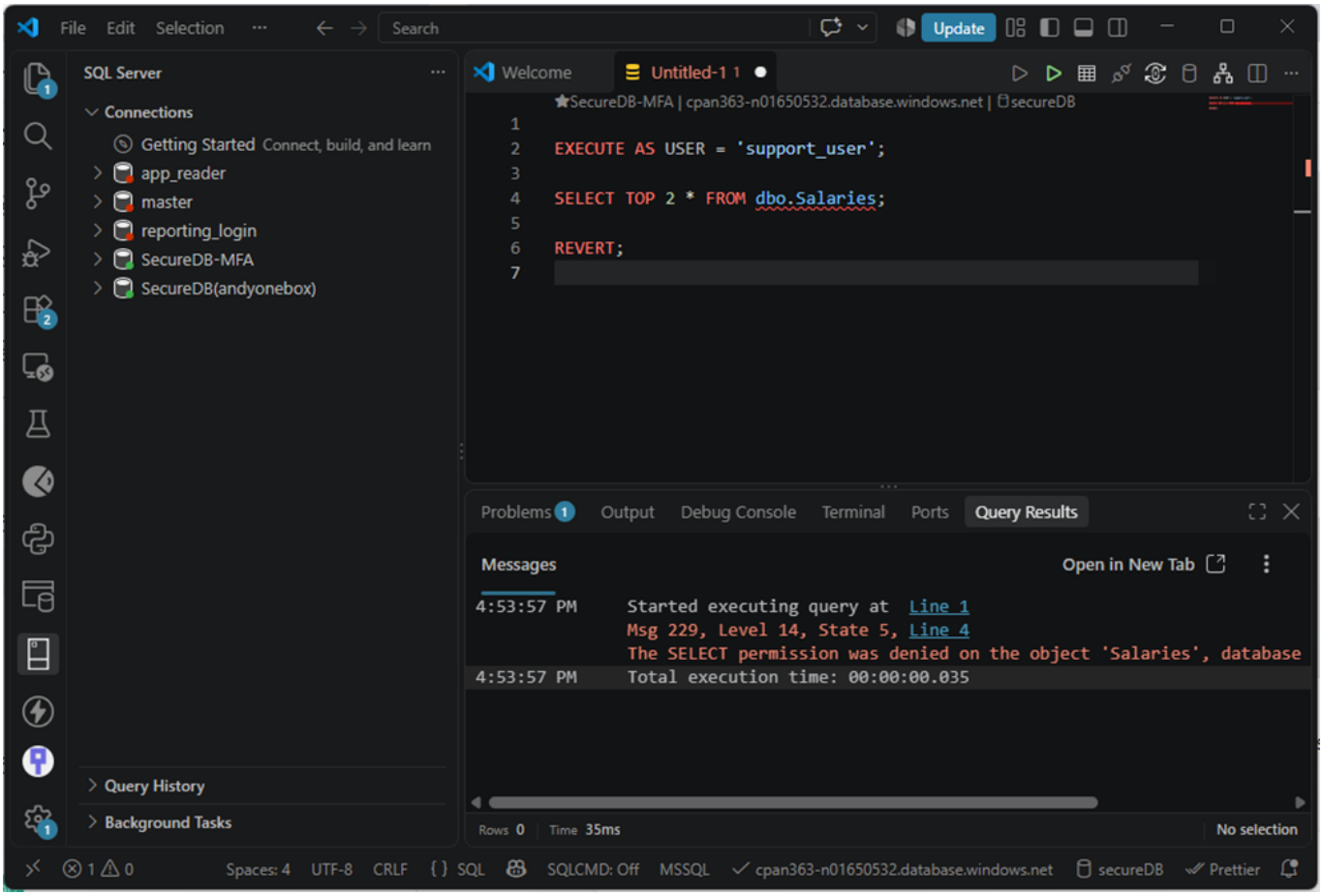


Figure 5. Support user receives error 229 when reading Salaries.

HR: All six tested SELECT, INSERT and UPDATE permissions across Employees and Salaries returned 1. Both SELECT queries returned two rows.

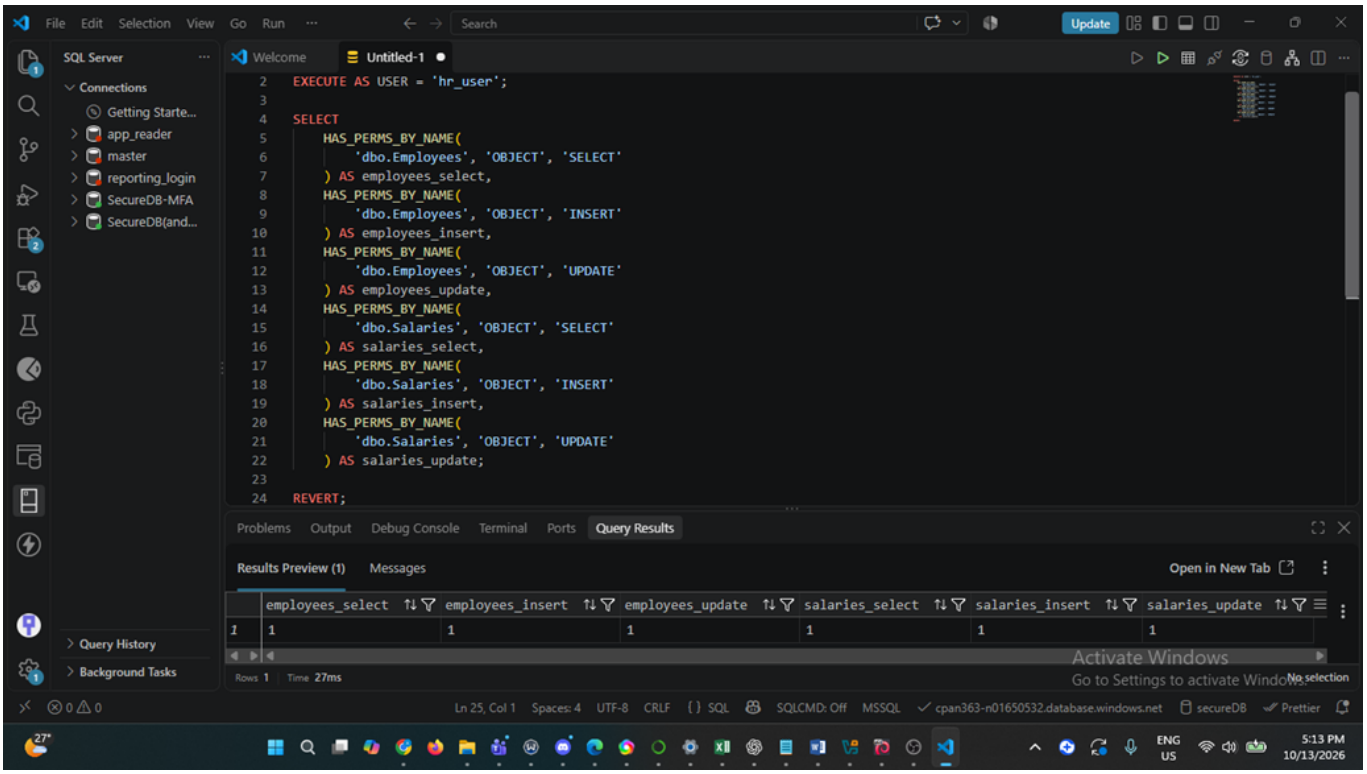


Figure 6. HR permission checks: all six permissions return 1.

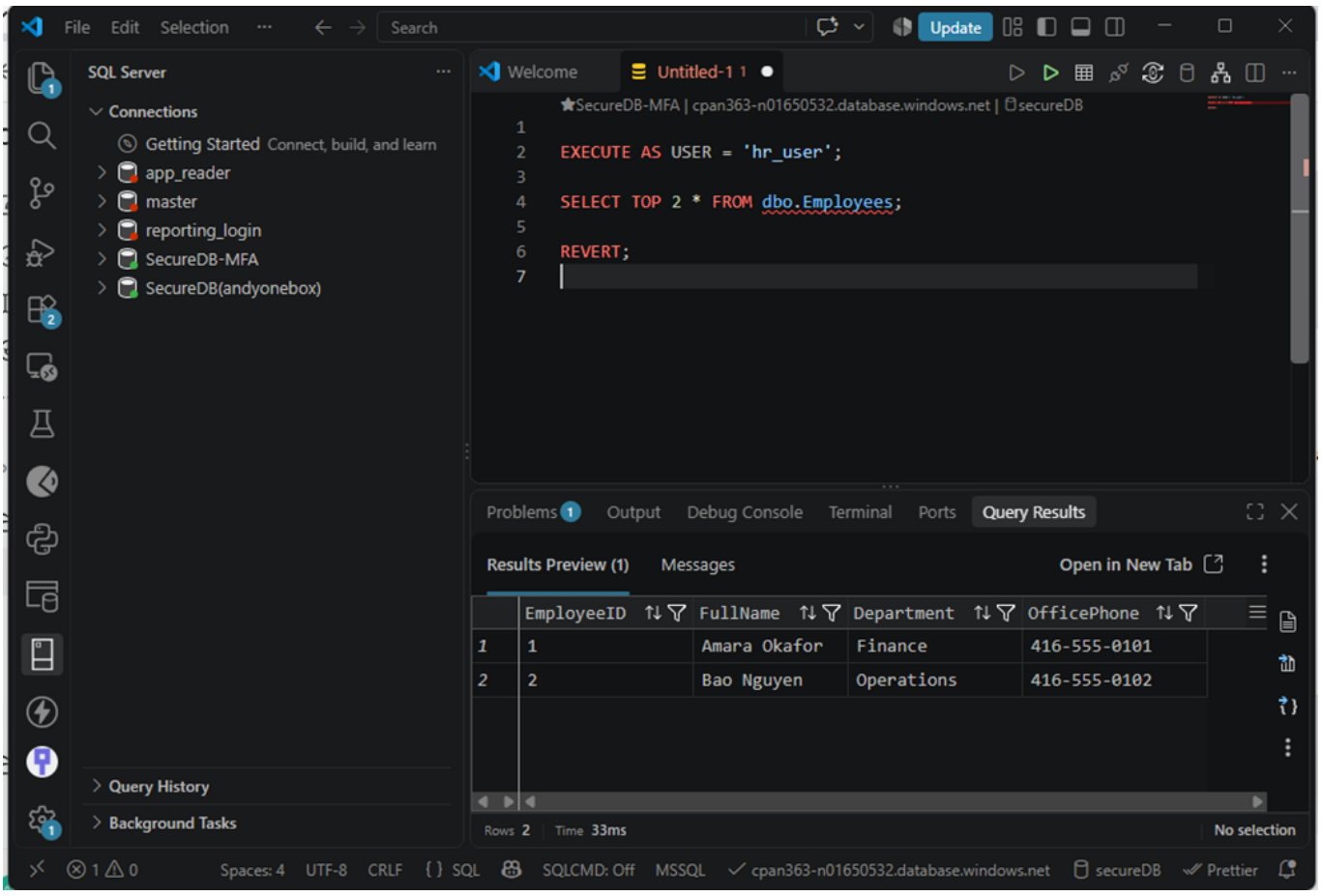


Figure 7. HR user successfully reads Employees.

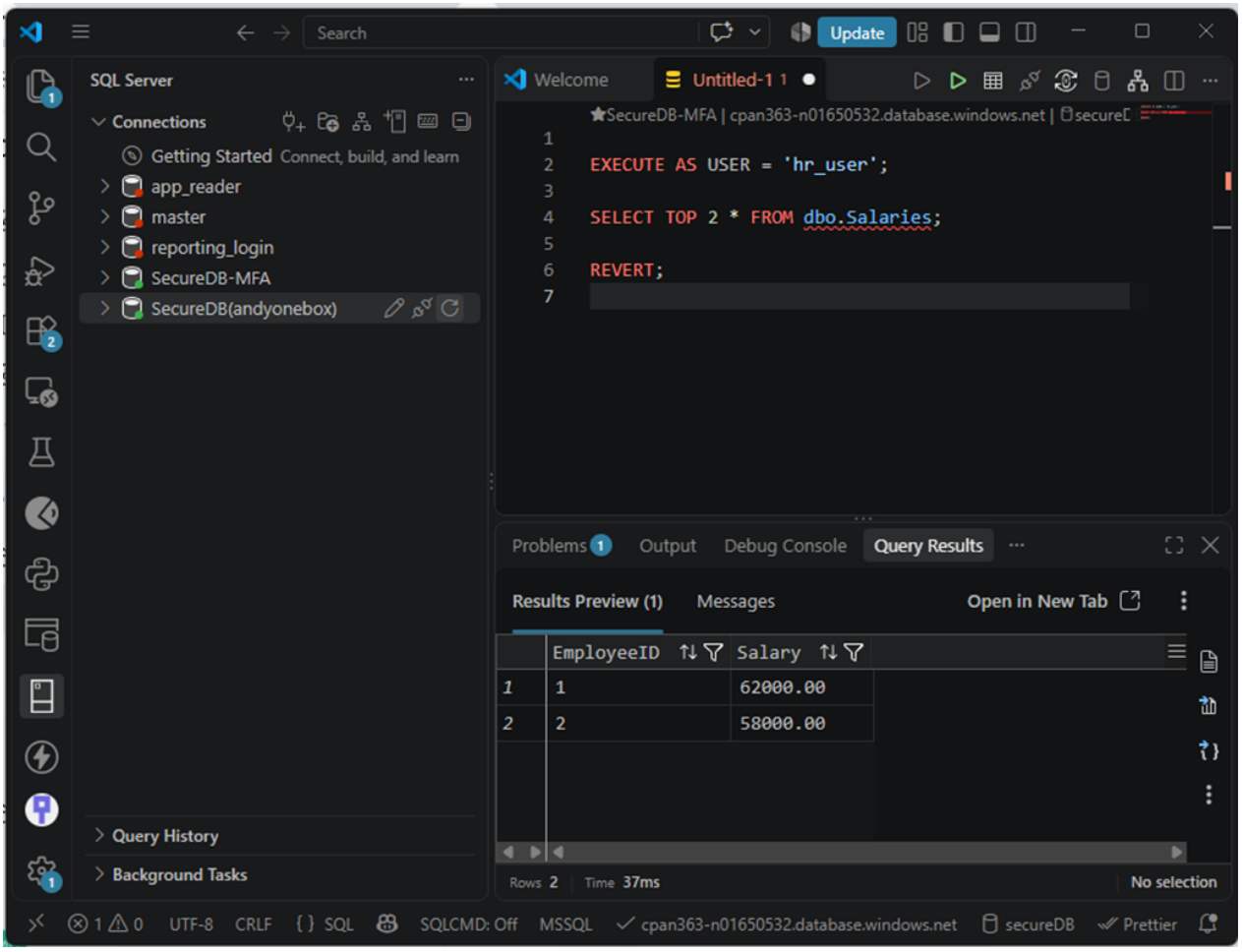


Figure 8. HR user successfully reads Salaries.

Analyst: Permission check returned 1 for Employees SELECT and 0 for Salaries SELECT. Reading Employees succeeded; reading Salaries produced error 229.

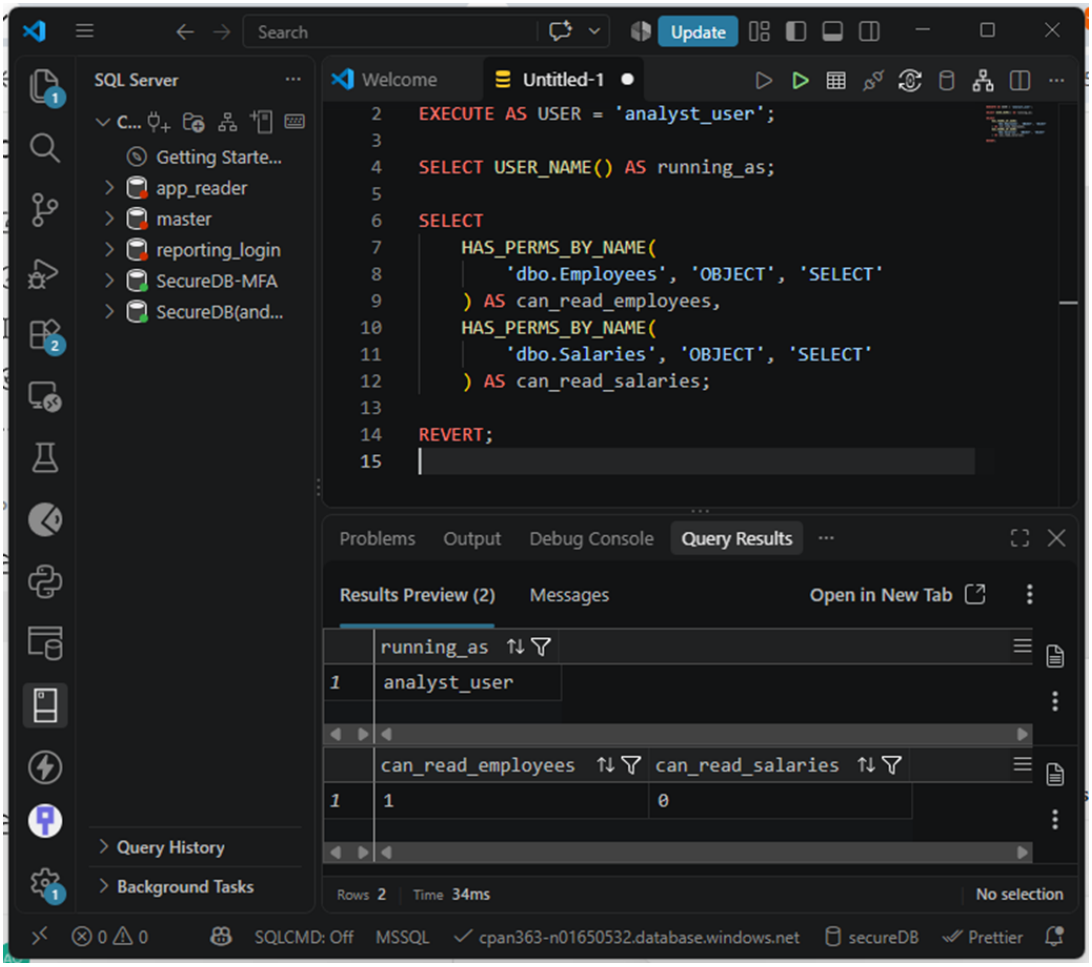


Figure 9. Analyst effective permission check: Employees = 1, Salaries = 0.

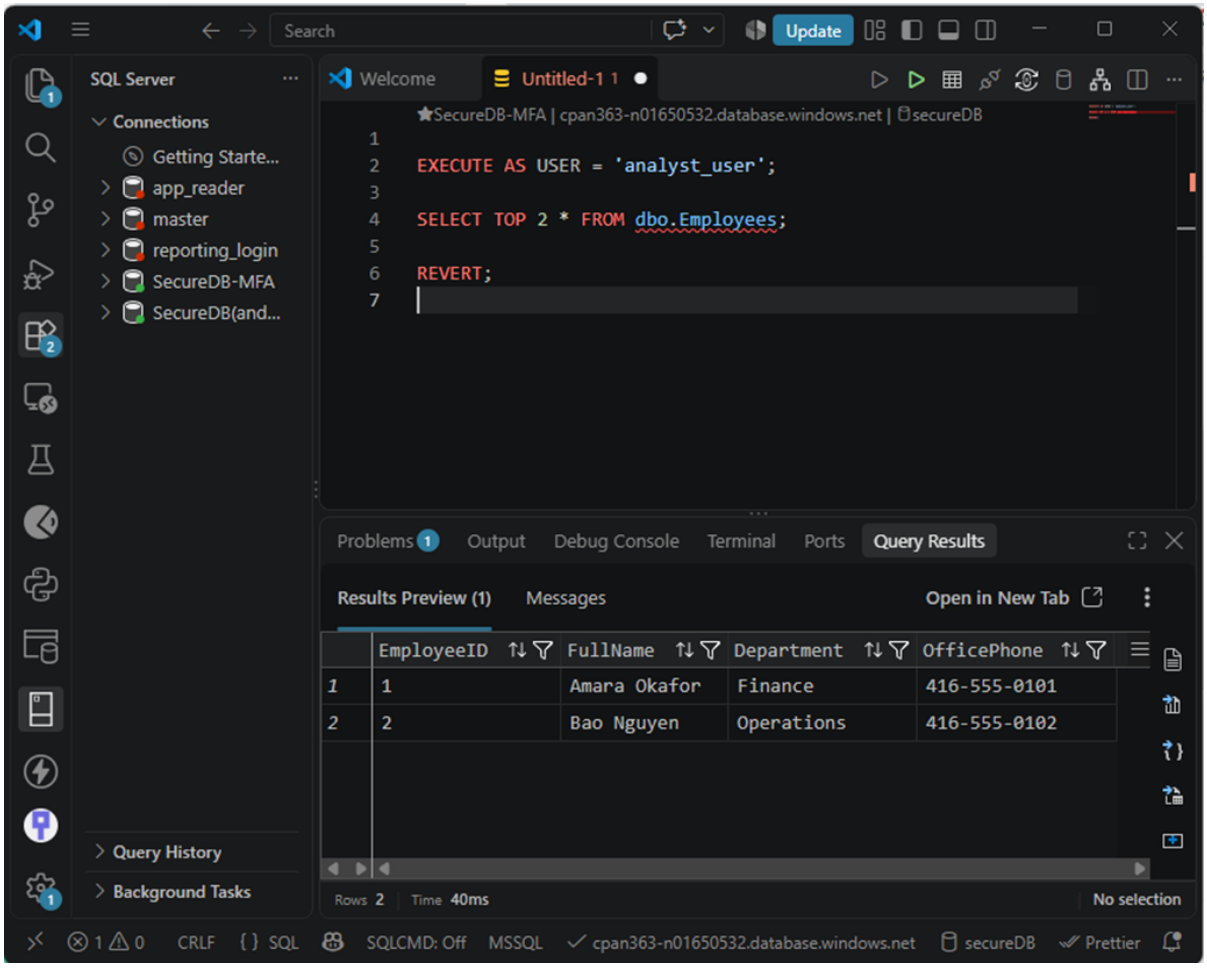


Figure 10. Analyst user successfully reads Employees.

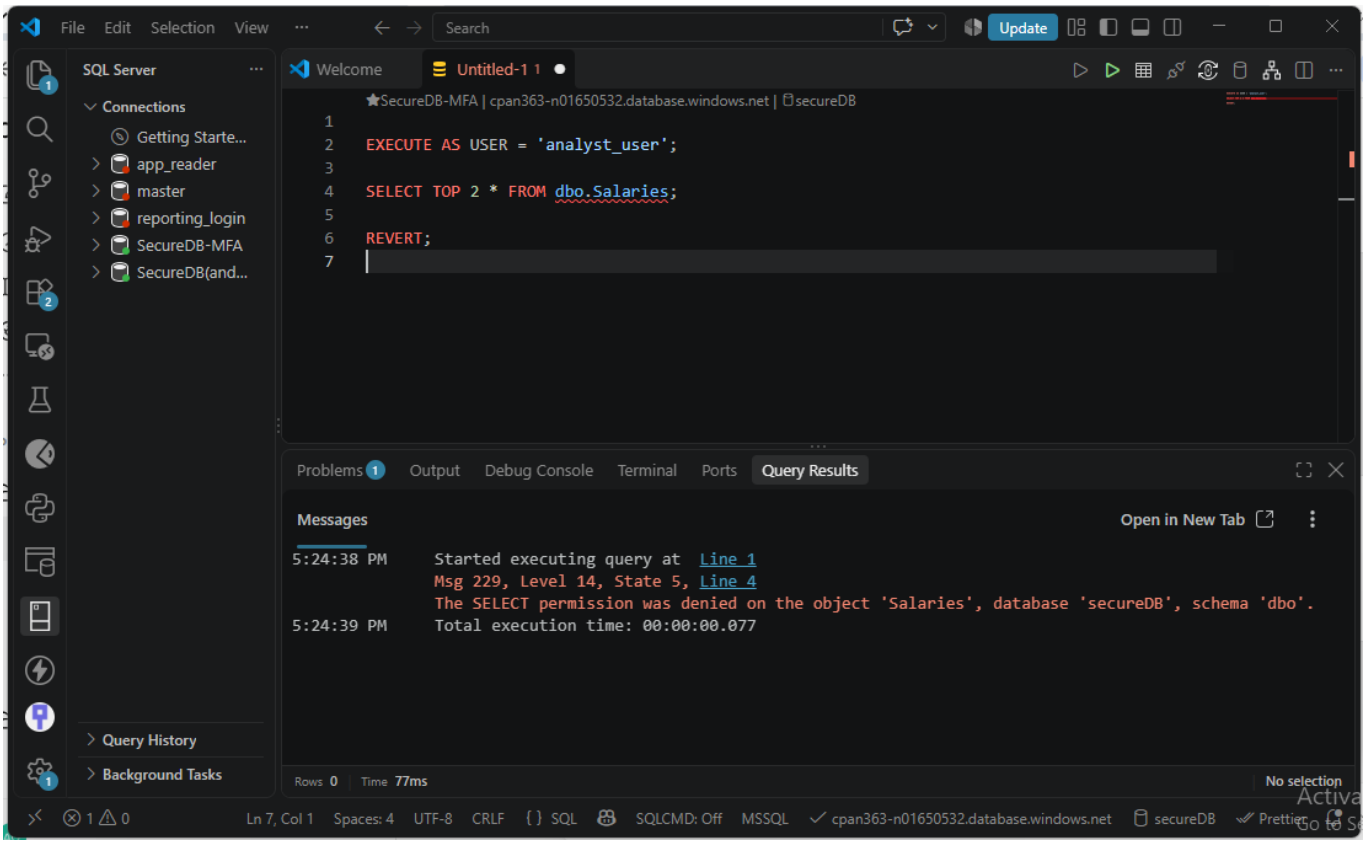


Figure 11. Analyst user is denied SELECT on Salaries.

Question 3.3 — Observation

The support and analyst users could both read employee records but not salary records, while HR could read both tables. Support has an explicit DENY on Salaries, whereas analyst has no salary SELECT grant. This distinction matters when permissions are inherited through additional roles.

Part 4 — Demonstrate DENY overrides GRANT

The lab test temporarily added support_user to db_datareader. Under support_user, HAS_PERMS_BY_NAME for Salaries SELECT returned 0, and a direct SELECT on Salaries failed with error 229. This is consistent with the explicit DENY on dbo.Salaries overriding the general read permission inherited from db_datareader. The temporary role membership was subsequently removed, as reflected in the final membership report.

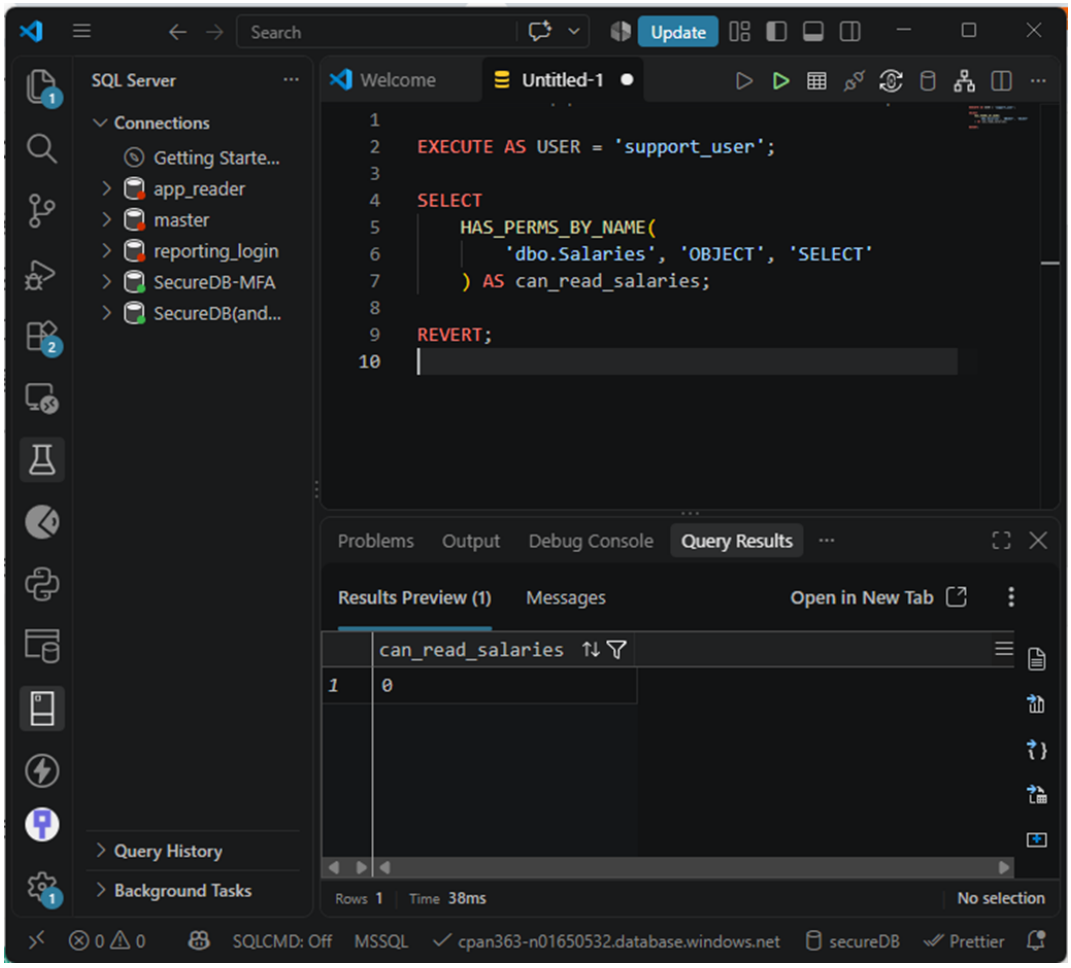


Figure 12. Salary permission check still returns 0 during the DENY-versus-GRANT experiment.

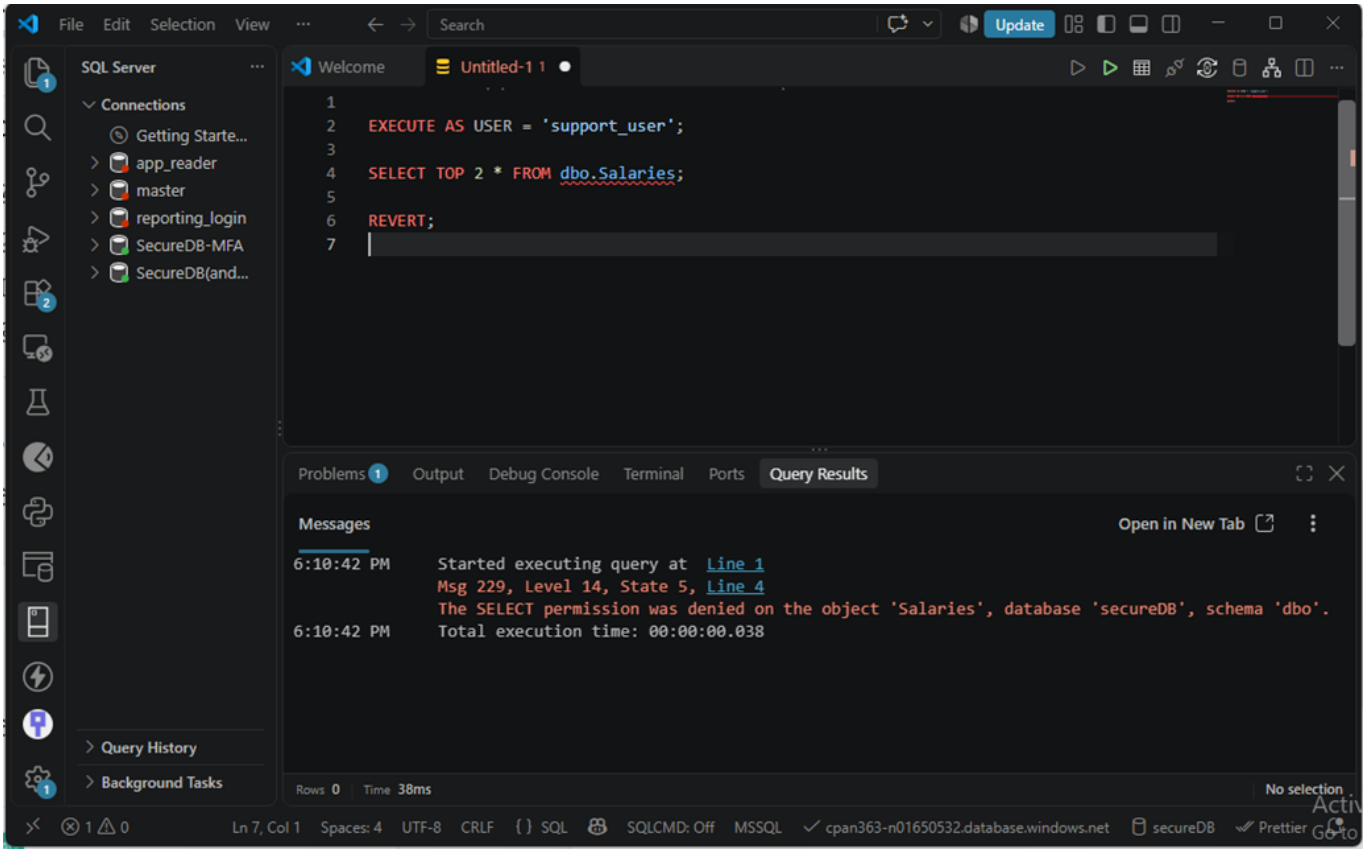


Figure 13. SELECT on Salaries still fails with error 229.

Question 4.3 — Explanation

The db_datareader role normally allows reading user tables. However, support_role explicitly denies SELECT on dbo.Salaries. Because the DENY takes precedence over the inherited GRANT in this test, support_user could not read the salary records.

Part 5 — Database security audit

The final membership report includes the three required custom role assignments and the existing db_owner membership. It does not show support_user as a db_datareader member.

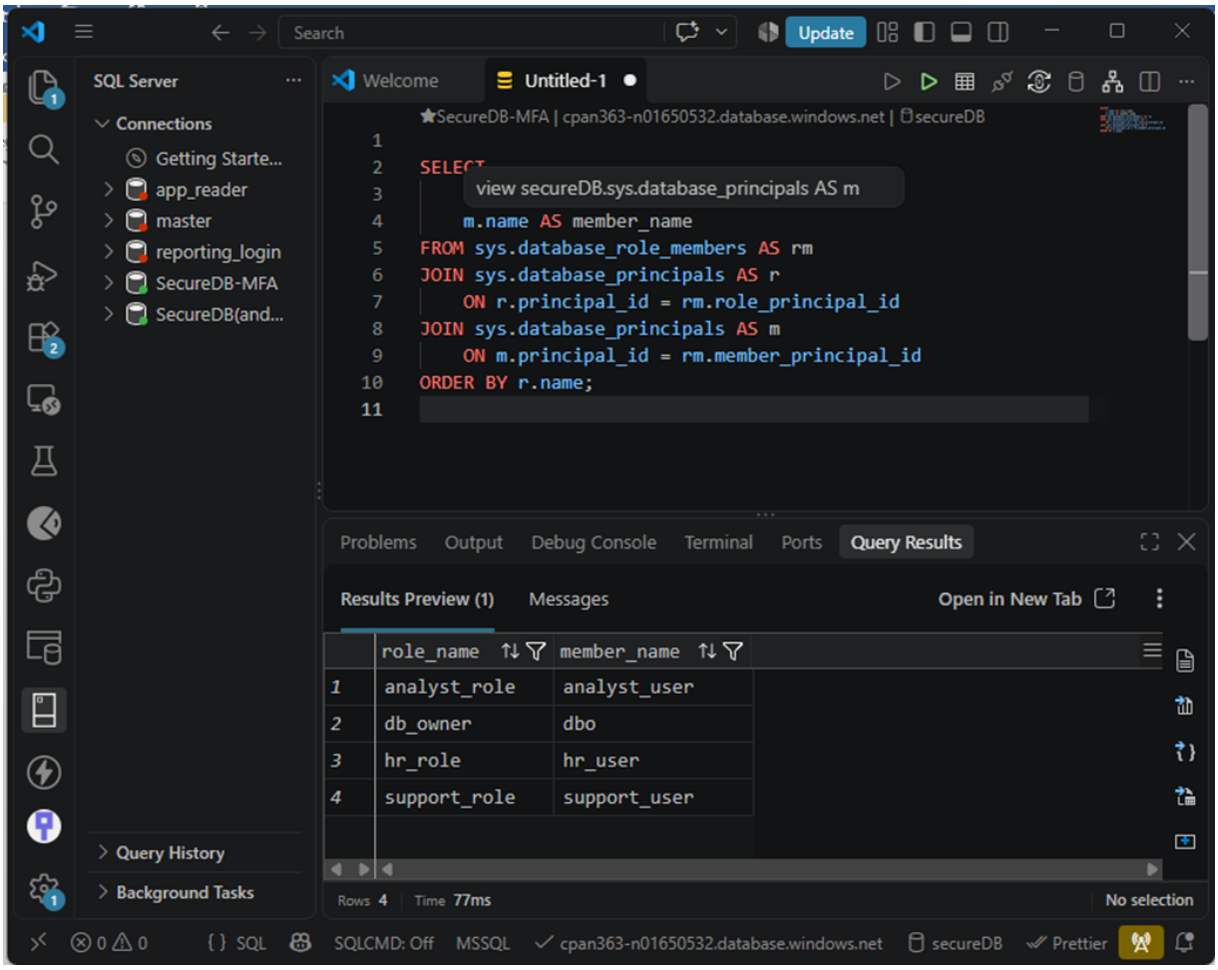
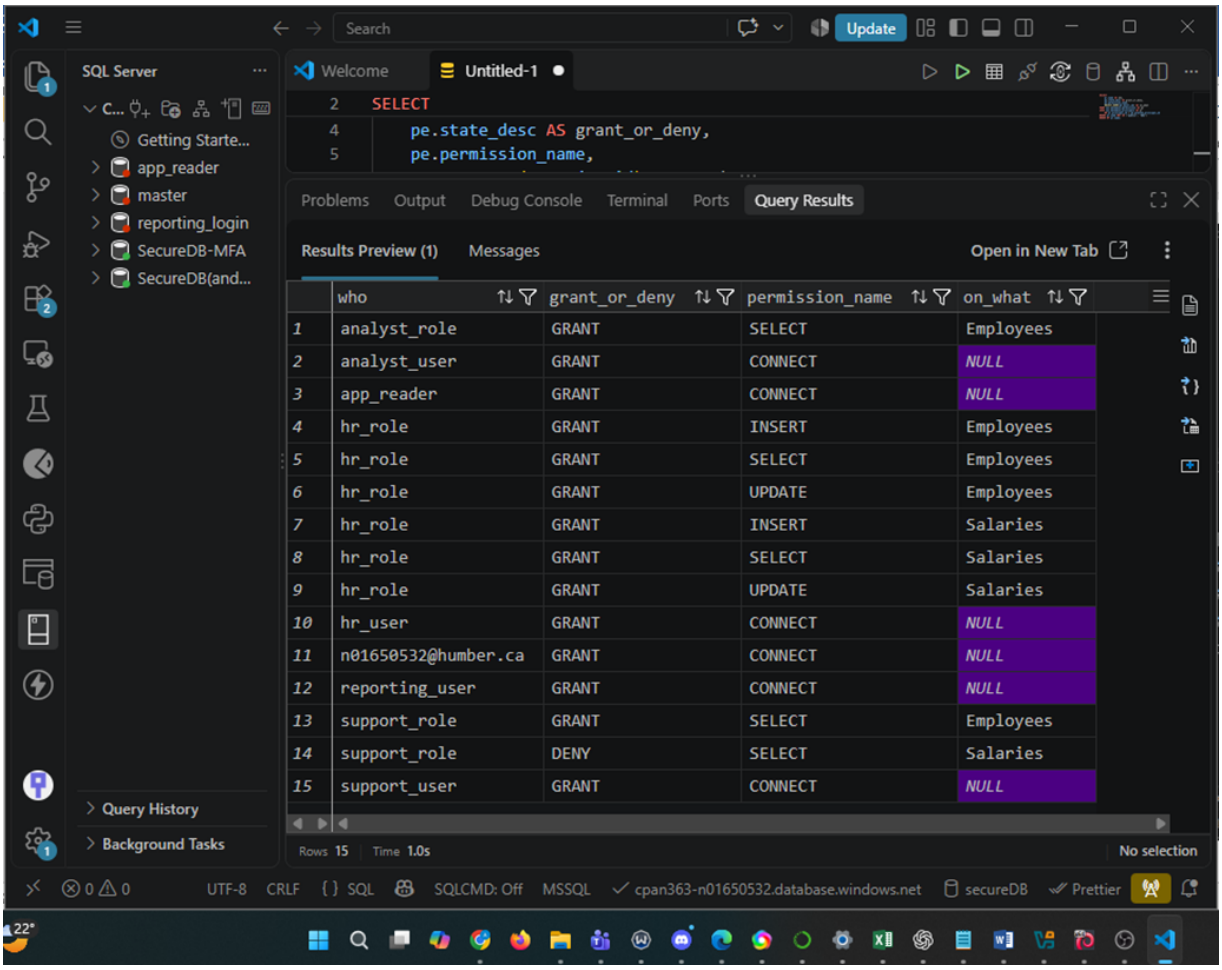


Figure 14. Final role membership report.

The permissions audit returned 15 rows, including nine role permissions required by the lab. The additional CONNECT entries are database-level user permissions, not extra table privileges assigned to the three roles.



	who	grant_or_deny	permission_name	on_what
1	analyst_role	GRANT	SELECT	Employees
2	analyst_user	GRANT	CONNECT	NULL
3	app_reader	GRANT	CONNECT	NULL
4	hr_role	GRANT	INSERT	Employees
5	hr_role	GRANT	SELECT	Employees
6	hr_role	GRANT	UPDATE	Employees
7	hr_role	GRANT	INSERT	Salaries
8	hr_role	GRANT	SELECT	Salaries
9	hr_role	GRANT	UPDATE	Salaries
10	hr_user	GRANT	CONNECT	NULL
11	n01650532@humber.ca	GRANT	CONNECT	NULL
12	reporting_user	GRANT	CONNECT	NULL
13	support_role	GRANT	SELECT	Employees
14	support_role	DENY	SELECT	Salaries
15	support_user	GRANT	CONNECT	NULL

Figure 15. Complete permissions audit, including support_role GRANT SELECT Employees and DENY SELECT Salaries.

Question 5.3 — Justification for each role permission

support_role — GRANT SELECT Employees: Allows support staff to look up employee contact and department details to resolve support requests.

support_role — DENY SELECT Salaries: Prevents support staff from accessing confidential compensation data outside their job duties.

hr_role — GRANT SELECT Employees: Allows HR to review employee records for personnel administration.

hr_role — GRANT INSERT Employees: Allows HR to register newly hired employees.

hr_role — GRANT UPDATE Employees: Allows HR to maintain employee records when details change.

hr_role — GRANT SELECT Salaries: Allows HR to review compensation records for payroll administration.

hr_role — GRANT INSERT Salaries: Allows HR to establish salary records for new employees.

hr_role — GRANT UPDATE Salaries: Allows HR to apply approved changes to employee compensation.

analyst_role — GRANT SELECT Employees: Allows analysts to read ordinary employee information for workforce reporting without salary access.

Question 5.4 — Request for db_owner access

I would ask the developer what issue they are diagnosing, which objects they need, and whether read-only access is sufficient. Instead of assigning db_owner, I would offer narrowly scoped, time-limited SELECT permissions on relevant tables or access to approved diagnostic views. I would review and revoke the additional access once the investigation is complete.

Submission and evidence notes

Companion file: N01650532_Lab3.sql. It contains the setup, grants, user impersonation tests, temporary db_datareader membership, cleanup and audit queries. Expected error-producing statements are annotated; execute them separately if reproducing the experiment. Screenshots show actual VS Code results rather than simulated output.